Neural Networks
COMP/EECE 7/8740

Final Project Report

Procedural Geographic Map Generation via Generative Adversarial Networks

Name: Jace King
ID: U00969076

May 1, 2026

Abstract

We train Wasserstein GANs with gradient penalty (WGAN-GP) and a DCGAN-style convolutional architecture to reproduce the distribution of 128×128 RGB terrain maps generated by layered simplex/Perlin noise. Seven generators, one per hand-tuned noise preset, were trained for 50 epochs each on 3,000 images per class. The unconditional models recover the macro-scale land/water layout for several presets but fail to reproduce the discrete six-tier terrain palette and exhibit single-feature mode collapse on three of the seven. A conditional variant (cDCGAN) over all seven presets simultaneously was attempted but did not finish within the available compute budget (epoch 14/50 at ∼76 min/epoch on CPU). We report training curves, qualitative samples, and quantitative evaluation of all seven generators against the held-out test set using FID, radial power-spectrum slope, palette color fidelity, and A*-based navigability metrics. The trained preset_01 generator is integrated into a real-time A* navigation game via a NumPy-only inference path with snap-to-palette post-processing.

1 Introduction

Procedural content generation in games has historically relied on hand-tuned noise functions, where a small parameter vector controls the visual character of an output. A generative model that learns this distribution end-to-end offers two practical advantages: (i) a single trained model can replace the parameter-tuning workflow with a latent-space sampling interface, and (ii) a conditional model can interpolate between or smoothly mix the qualitative "flavors" of distinct presets, which a hand-coded generator cannot do without explicit blending logic.

We restrict the problem to a tractable supervised setting: the data distribution itself is fixed and known (layered simplex/Perlin noise with a piecewise-constant terrain palette), the resolution is moderate (128×128), and the seven terrain types are well separated in parameter space. Our goal is to train a GAN that reproduces this distribution well enough for its output to be used as drop-in terrain maps in a simple navigation game, and to evaluate the degree to which the learned distribution matches the real one using both visual and quantitative criteria.

2 Methodology

We frame terrain generation as an unconditional generative modeling problem: train a generator G to map Gaussian noise $\mathbf{z}$ to synthetic terrain images whose distribution is indistinguishable from layered Perlin-noise maps. For each of the seven hand-tuned noise presets we train a separate unconditional model; a seventh, conditional model is trained jointly over all presets. We use the Wasserstein GAN with gradient penalty (WGAN-GP) [**?**] rather than vanilla DCGAN [**?**] because of its superior training stability and interpretable critic loss.

2.1 Loss Function

We use the WGAN-GP objective [**?**]:

$$\mathcal{L}_D = E[D(G(\mathbf{z}))] - E[D(\mathbf{x})] + \lambda\, E\big[(\|\nabla_{\hat{\mathbf{x}}} D(\hat{\mathbf{x}})\|_2 - 1)^2\big],$$
$$\mathcal{L}_G = -E[D(G(\mathbf{z}))],$$

with $\hat{\mathbf{x}} = \alpha\mathbf{x} + (1-\alpha)G(\mathbf{z})$, $\alpha \sim \text{Uniform}[0,1]$, and $\lambda = 10$. We run $n_{\text{disc}} = 5$ critic updates per generator update.

2.2 Optimization

Adam with $\eta = 2{\times}10^{-4}$, $\beta_1 = 0.5$, $\beta_2 = 0.999$ for both networks. Batch size 64, seed 42 for the data shuffle and the fixed evaluation $\mathbf{z}$. The original WGAN-GP paper recommends $\beta_2 = 0.9$; we used the Adam default of 0.999 and observed no divergence in any of the eight runs, but a re-run with $\beta_2 = 0.9$ would be the more conservative choice.

2.3 Compute

Training was performed on a SLURM cluster, one node per preset, 16 CPUs and 32 GB RAM per task, no GPU available (`tf.config.list_physical_devices('GPU')` returned `[]`). Per-epoch wall time averaged ∼580–700 s (∼10–12 min) for the unconditional models and ∼4,600 s (∼76 min) for the conditional model; total wall time was ∼8–10 h per unconditional run.

3 Model Description

Following DCGAN conventions [**?**], the generator G maps a 100-dimensional latent vector $\mathbf{z} \sim \mathcal{N}(0, I)$ to a 128×128×3 image with tanh output. The discriminator (critic) D maps an image to an unbounded scalar.

3.1 Generator (4.83 M params)

$\mathbf{z} \in R^{100} \to$ Dense($8{\cdot}8{\cdot}512$) $\to$ Reshape$(8, 8, 512)$ $\to$ BN $\to$ LReLU(0.2), then three upsampling blocks

$$\big[\text{UpSample2x} \to \text{Conv2D}(3{\times}3) \to \text{BN} \to \text{LReLU}(0.2)\big]$$

with channel widths $\{256, 128, 64\}$ (8→16→32→64), followed by a final UpSample2x → Conv2D($3{\times}3$, tanh) to 3 channels at 128×128.

3.2 Critic (2.79 M params)

Four strided conv blocks Conv2D(4×4, stride 2) → LReLU(0.2) with widths $\{64, 128, 256, 512\}$ (128→64→32→16→8) then Flatten → Dense(1). We use *no normalization* in the critic: BatchNorm couples samples within a minibatch and is incompatible with the per-sample gradient penalty.

3.3 Conditional Variant (cDCGAN)

For the cDCGAN, a 7-dim one-hot label is concatenated to $\mathbf{z}$ before the generator's first Dense layer, and to the critic's flattened feature vector before its final Dense layer. All other architectural choices are unchanged; the conditioning adds ∼5% to the parameter count.

4 Experiments and Results

4.1 Database

The training dataset consists of 21,000 images: 3,000 per preset for seven presets. Each image is 128×128 RGB, `uint8`. A held-out test set of 1,400 images (200 per preset) is reserved for evaluation. All images are produced by `generate_dataset.py` from layered simplex noise via `noise2array`.

Presets. Each preset fixes a base (depth, scale, factor, persistence) tuple. To prevent the GAN from memorizing a small template set, every image samples (i) a random spatial offset $(x_0, y_0) \in [0, 9999]^2$ in noise space and (ii) multiplicative jitter on the parameters: scale ±15%, persistence ±10%, factor ±10%.

Color palette. The continuous noise field $n \in R$ is normalized to $[0, 1]$ and quantized through six fixed thresholds into the categorical palette {deep water, shallow water, sand, plains, mountains, mountain tops}. This piecewise-constant mapping is the most challenging structural feature of the data for a tanh-output convolutional generator to reproduce, as discussed in Section **??**.

Figure 1: One representative real sample per preset. Note the categorical color palette and the highly preset-specific land/water ratios.

Reference samples. Figure **??** shows one representative sample from each preset.

4.2 Training and Testing Logs

Training curves

Figure **??** plots the per-epoch mean $\mathcal{L}_G$ and $\mathcal{L}_D$ for each preset from `logs/dcgan_preset_*.csv`, rendered by `scripts/plot_loss_curves.py`.

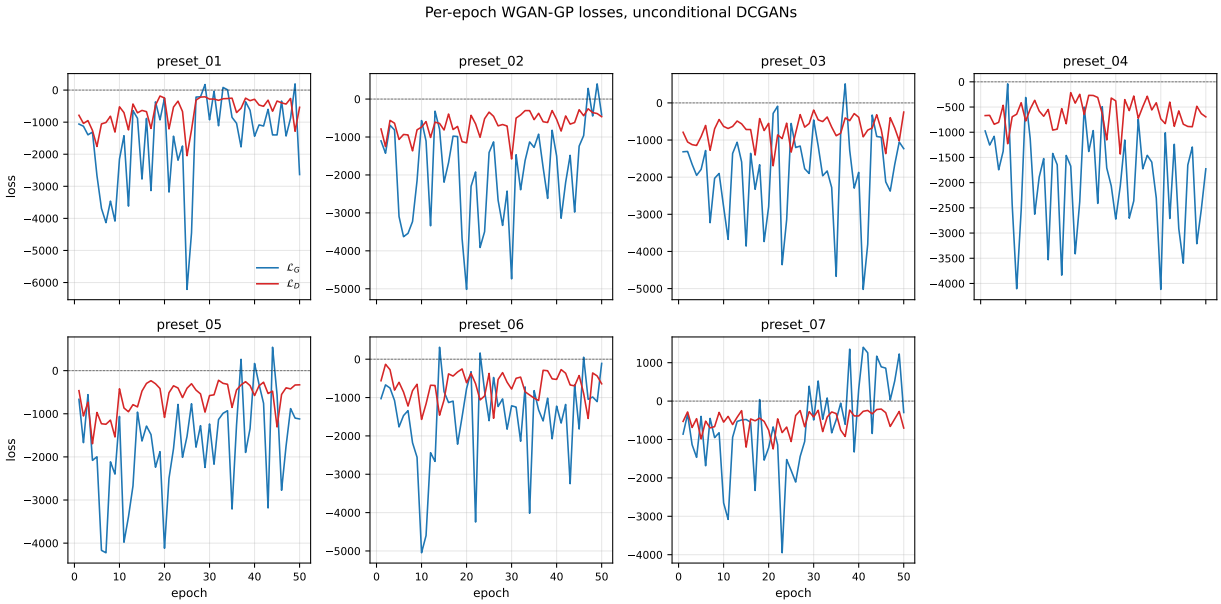


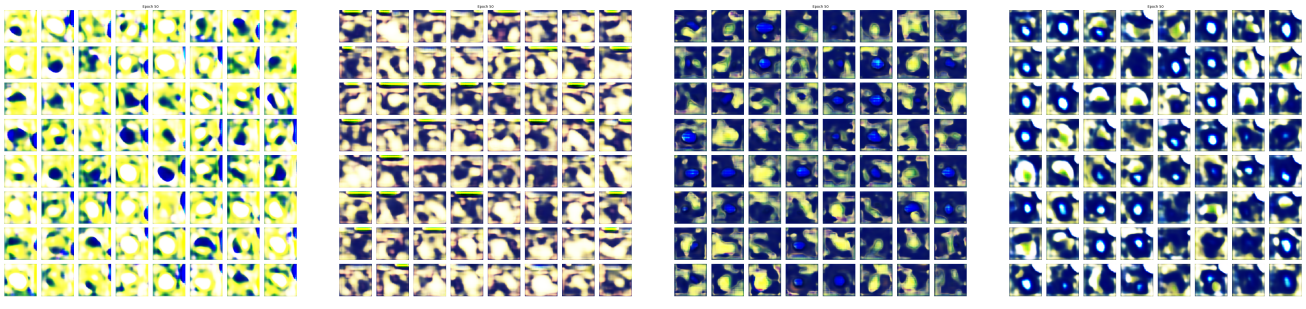
Figure 2: Per-epoch generator and critic losses for each unconditional preset over 50 epochs of WGAN-GP training.

The Wasserstein critic loss is unbounded by construction; large negative values and substantial epoch-to-epoch variance are expected and do not by themselves indicate failure. Across all seven presets the magnitude of both $\mathcal{L}_G$ and $\mathcal{L}_D$ is large in the first $\sim$10 epochs (reflecting an under-converged critic), then settles into an oscillating but bounded regime for the rest of training. Final-epoch values for $\mathcal{L}_G$ range from $\sim -2{,}600$ (preset_01) to ~ -110 (preset_06); these absolute values are not directly comparable across presets because the critic for each model is calibrated to its own data distribution. We do not observe the diverging-loss / saturated-output failure mode characteristic of vanilla DCGAN; the gradient penalty appears to keep the critic in the 1-Lipschitz neighborhood throughout.

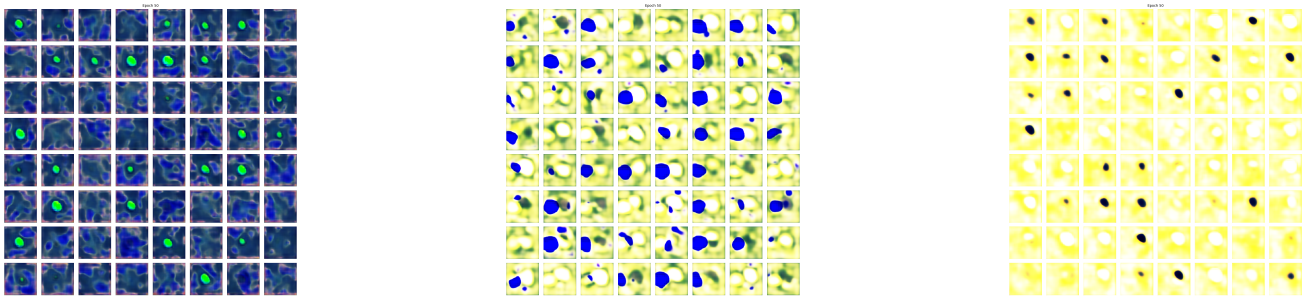
Generated samples

Figure **??** shows the final-epoch fixed-noise sample grid for each preset. All seven grids use the same $\mathbf{z}$ seed (`tf.random.normal((64, 100), seed=0)`), so within-grid variation reflects sensitivity to $\mathbf{z}$ for

that generator.



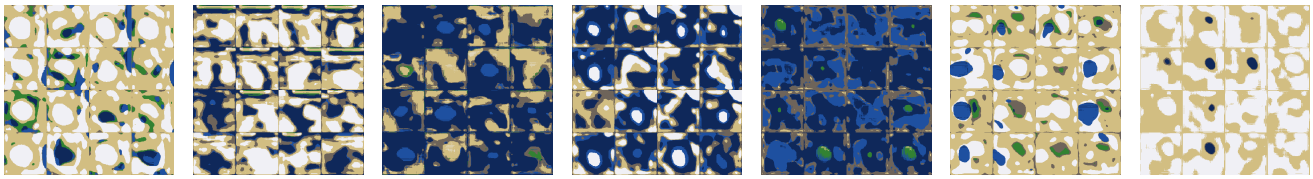
(a) preset_01 (b) preset_02 (c) preset_03 (d) preset_04



(e) preset_05 (f) preset_06 (g) preset_07

Figure 3: Generator output at epoch 50, 64 samples per preset (same **z** seed across all presets).

Figure **??** shows the same generators' output after snap-to-palette post-processing.



preset_01 preset_02 preset_03 preset_04 preset_05 preset_06 preset_07

Figure 4: Generator output after snap-to-palette post-processing (4×4 samples per preset, same **z** seed as Figure **??**). Snapping maps each pixel to its nearest palette entry by Euclidean distance in RGB space, eliminating the smooth color interpolation and producing maps directly consumable by the game's speed-map lookup.

Quantitative evaluation

Table **??** summarizes four quantitative metrics computed via `scripts/evaluate.py` against the 200-image held-out test split for each preset, using the `G_final.weights.h5` checkpoint. Generated samples are n=200 images drawn from $\mathbf{z} \sim \mathcal{N}(0, I)$, seed 0.

Conditional DCGAN

The conditional model was trained on all 21,000 images simultaneously with a 7-dim one-hot label conditioning both networks. On the same 16-CPU node, per-epoch wall time was ∼76 minutes (vs ∼10–12 minutes for the unconditional models on a single preset's 3,000 images), driven primarily by the 7× larger dataset size. The run reached epoch 14 of the planned 50 before the project deadline; only the epoch-10 checkpoint and sample grids at epochs 5 and 10 (`samples/cdcgan/`) were saved (sampling

Table 1: Quantitative evaluation: 200 held-out real images vs. 200 generated samples per preset. β_{real} is the mean log-log slope of the radially-averaged 2D power spectrum for real images; $\Delta\beta = \beta_{\text{gen}} - \beta_{\text{real}}$ (closer to 0 is better). Pal. RMSE is mean per-pixel Euclidean distance to the nearest palette prototype in RGB space (real = 0 by construction; lower = better). Tortuosity and reachable fraction are from A* traversal on the binary passable mask.

		PSD slope			Navigability (gen)	
Preset	FID↓	β_{real}	$\Delta\beta$↑	Pal. RMSE↓	Tortuosity	Reachable
preset_01	385	−2.61	−0.66	62.9	1.42	0.92
preset_02	424	−2.53	−0.81	43.6	1.53	0.91
preset_03	411	−2.60	−0.59	36.5	1.34	0.63
preset_04	370	−2.63	−0.83	41.4	1.54	0.60
preset_05	378	−2.62	−0.41	38.8	1.44	0.35
preset_06	372	−2.63	−0.63	59.4	1.45	0.98
preset_07	340	−2.50	−0.29	64.1	1.56	0.82

and checkpointing intervals did not align with the final epoch). The grids show the same coarse blob structure as the early-epoch unconditional models, with no clear visual separation between the seven label conditions yet – consistent with an under-trained conditional generator that has not yet specialized per class.

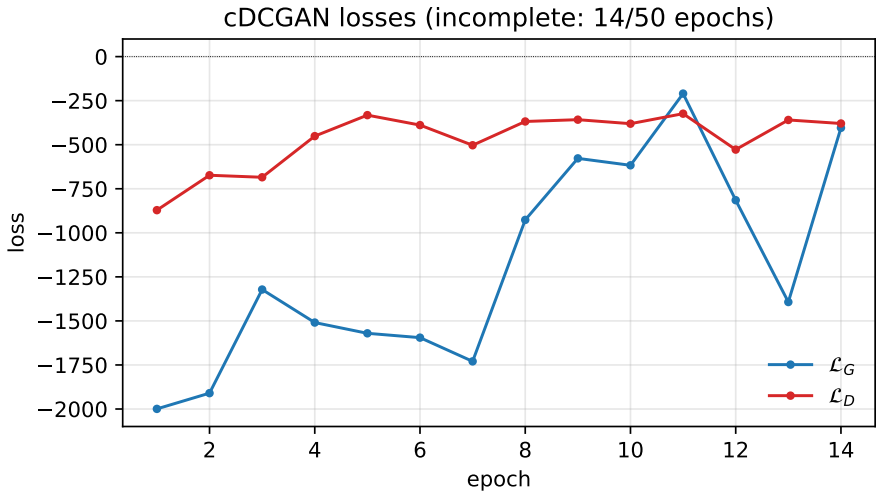


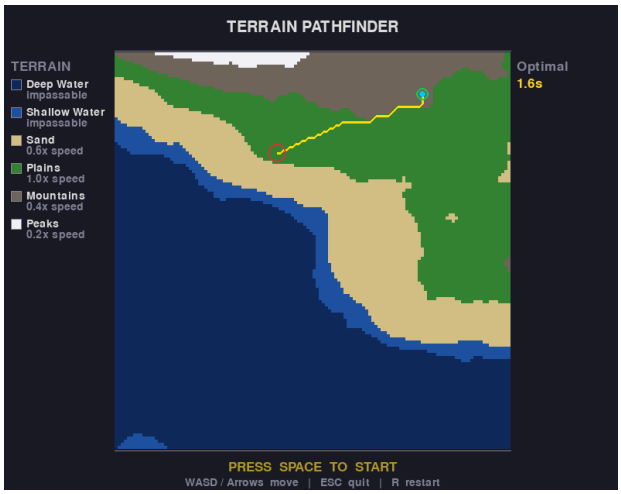
Figure 5: cDCGAN per-epoch generator and critic losses over the 14 completed epochs.

Game integration

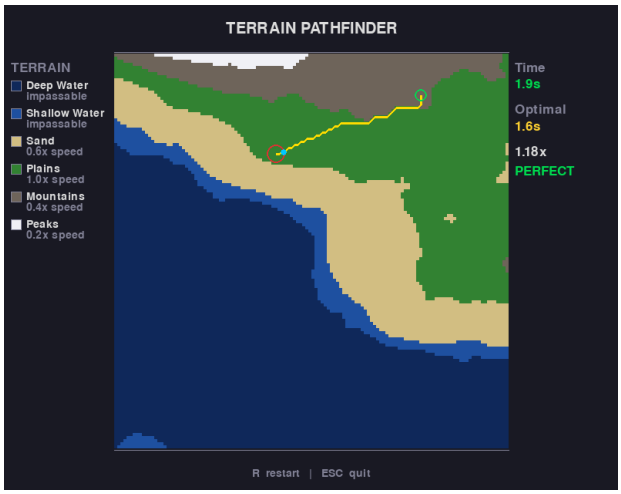
We built a small `pygame` navigation game (`game/game.py`) in which the player walks from a green start to a red goal across the generated 128×128 terrain map; deep and shallow water are impassable and the remaining four palette tiers map to per-cell movement-speed multipliers $\{0.6, 1.0, 0.4, 0.2\}$. An A* solver computes the optimal traversal time on the same speed grid and the in-game timer is graded against it.

The game accepts `--source perlin` (default; calls `noise.create_noise_map` directly) or `--source gan --preset preset_NN`, in which case it loads an exported `generator.npz` from `checkpoints/<preset>/`. The exported weights are produced by `scripts/export_generator.py`; `scripts/gan_terrain.py` then performs the entire forward pass in NumPy (BatchNorm, 3×3 convolutions via `im2col`, upsampling, tanh), so the game has no TensorFlow dependency. The raw tanh output is mapped to $[0, 255]$, then each

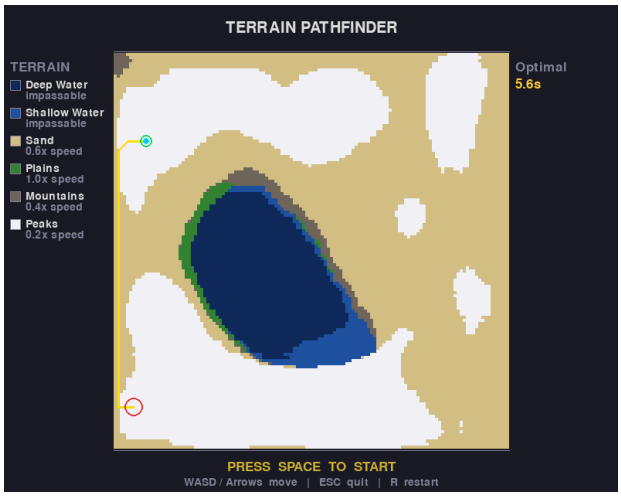
pixel is snapped to its nearest-Euclidean-RGB palette entry; the same index lookup yields the discrete speed map. All seven unconditional generators have been exported; the partial cDCGAN checkpoint was not exported.



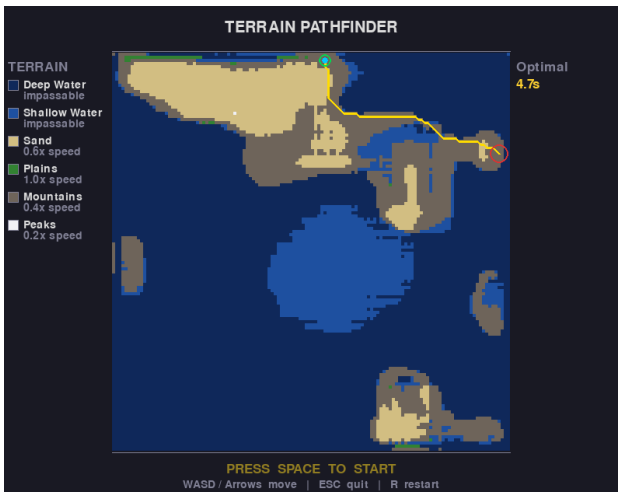
(a) Perlin terrain at game start. The yellow overlay is the A* optimal path; the UI shows optimal traversal time of 1.6 s before the player moves.



(b) Same Perlin terrain after goal reached. Actual time 1.9 s (1.18× optimal); graded "PERFECT".



(c) GAN terrain (`--preset preset_01`). Sand/peak mode collapse; A* optimal time 5.6 s due to slow terrain.



(d) GAN terrain (`--preset preset_03`). Single oval water body; A* path (4.7 s) circumnavigates the impassable water.

Figure 6: In-game screenshots. Top row: Perlin-noise source (default). Bottom row: GAN source. The terrain legend, A* path, player position, and real-time score are rendered by `game/game.py`.

4.3 Discussion and Comparison

Per-preset qualitative analysis.

preset_02, preset_06

These two recover the macro-scale structure most convincingly. The generator produces plausible land/water boundaries with reasonable shape diversity across $\mathbf{z}$, and the dominant palette colors (deep navy water, light cream/sand) are present. Higher palette tiers (mountains, mountain tops) are not visibly recovered; transitions are smoothed rather than thresholded.

preset_03, preset_04

These show partial structure but a clear *single-feature* mode: nearly every preset_04 sample places one bright island near the center of a dark sea; preset_03 places one elongated water body in a noisier yellow-green field. Within-grid diversity exists in shape and position but not in feature count, which is a textbook mode-collapse signature.

preset_01, preset_05, preset_07

These collapsed most severely. The output is dominated by a single background color (yellow/cream for 01 and 07; deep blue for 05) with at most one localized blob of contrast. The overall color distribution is qualitatively wrong relative to the training data: preset_01 training samples are dominated by water and plains green, but the generator has settled on a sand-yellow background.

Why the discrete palette is hard. The Perlin-noise data has a piecewise-constant color field with sharp boundaries at the six fixed thresholds. A tanh-output convolutional generator must learn six near-step-function responses in RGB space conditional on the underlying smooth noise field it is implicitly synthesizing. With LReLU activations and BatchNorm, gradient flow naturally favors smooth output ramps; we observe exactly this – the generator interpolates between palette colors rather than thresholding them. Snap-to-palette post-processing at inference time (Figure **??**) closes most of this gap visually.

Comparison with the real (Perlin-noise) distribution. The real Perlin-noise maps serve as the baseline; all quantitative metrics compare generated distributions to the 200-image held-out test split.

FID. FID (Table **??**) ranges from 340 to 424. These values are high by ImageNet-GAN standards, expected given the small evaluation set (200 samples per side) and the mismatch between InceptionV3 features and synthetic terrain. The FID ranking does *not* agree with the qualitative assessment: preset_07 achieves the lowest FID (340) despite being visually collapsed, while the strongest visual generator (preset_02) has the highest FID (424). This reversal suggests the InceptionV3 embedding is a poor proxy for the terrain structure we care about. The power-spectrum and navigability metrics below are more informative.

Power-spectrum slope. Real terrain images yield $\beta \approx -2.5$ to -2.6, consistent with the $1/f^2$ power law of layered Perlin noise. All seven generators produce steeper spectra ($\Delta\beta < 0$, range -0.29 to -0.83), meaning generated images over-emphasize large-scale structure and suppress the mid- and high-frequency detail present in the training data. preset_07 deviates least ($\Delta\beta = -0.29$); preset_04 deviates most ($\Delta\beta = -0.83$), consistent with its strong single-feature mode collapse. Figure **??** shows the mean radial power spectra for all seven presets.

Palette RMSE. Real images score 0.0 by construction. Generated palette RMSE ranges from 36.5 (preset_03) to 64.1 (preset_07), confirming that all generators interpolate between palette colors rather than learning to threshold them. Palette RMSE does not correlate strongly with FID: a generator that collapses to one dominant palette color can achieve low RMSE while producing poor spatial statistics.

Navigability. Real images score tortuosity ≈ 1.27–1.30 and reachable fraction ≈ 0.91–0.98. Generated maps show uniformly higher tortuosity (1.34–1.56) but more variable reachable fraction. preset_06 closely matches both real axes (tortuosity 1.45, reachable 0.98), consistent with its strong visual quality. preset_05 collapses to the lowest reachable fraction (0.35): the generator produces a map dominated by impassable water with one small island, leaving most passable area disconnected. presets 03 and 04 similarly show low reachable fraction (0.63 and 0.60) due to single-feature collapse.

What worked. WGAN-GP trained stably for all eight runs; no run diverged or required restart. The macro-scale spatial statistics are recovered for at least two presets. End-to-end integration into the navigation game (NumPy inference, snap-to-palette, A* solver, in-game scoring) works for all seven generators.

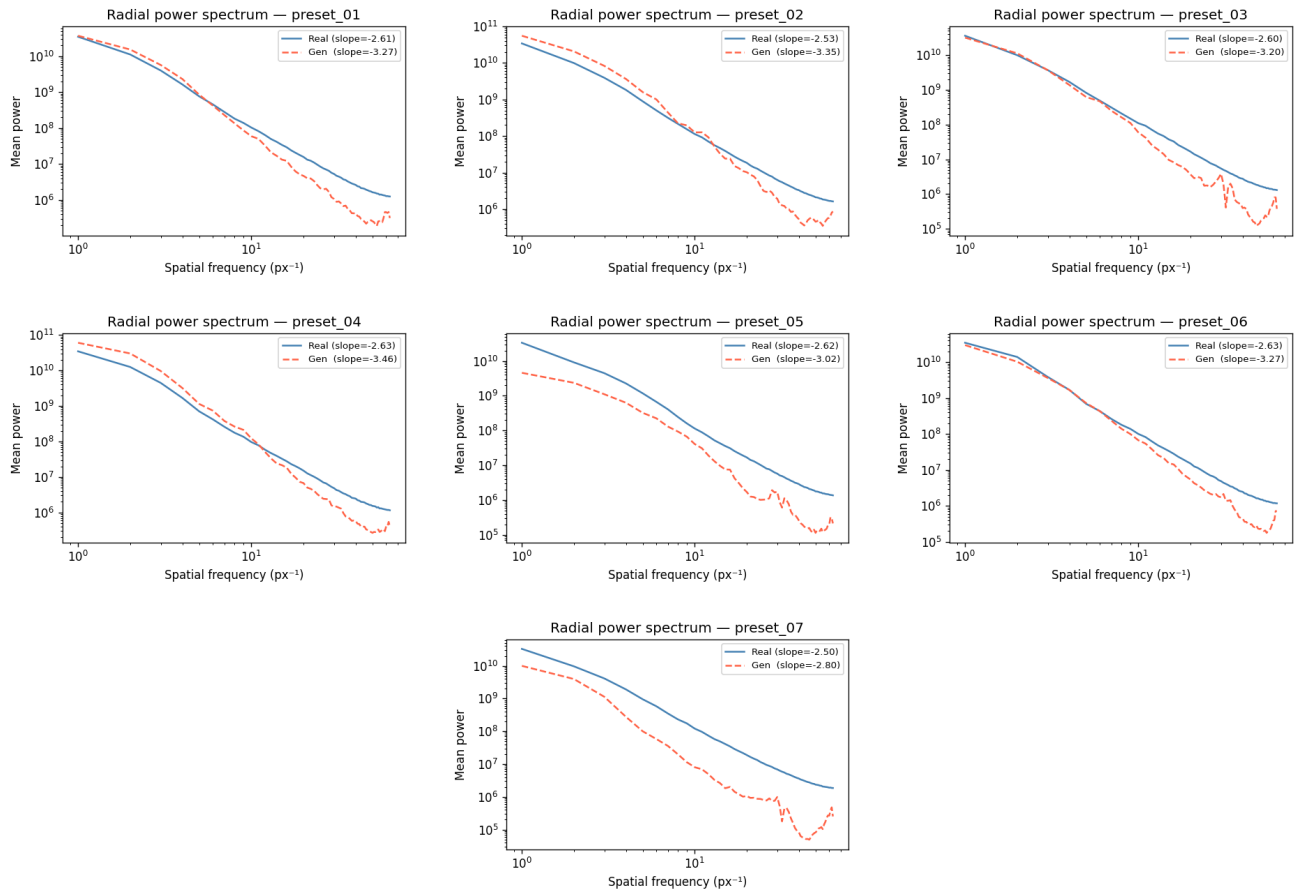


Figure 7: Radially-averaged 2D power spectra (mean over 200 real and 200 generated images per preset, log-log axes). Generated images have uniformly steeper slopes, reflecting the generator's tendency to produce large smooth blobs rather than the multi-scale detail of Perlin noise.

What didn't. (i) The discrete six-tier palette is not learned – outputs interpolate between colors rather than thresholding them. (ii) Three of seven generators show clear single-feature mode collapse. (iii) The cDCGAN did not finish training on CPU. (iv) FID correlates poorly with visual quality for this data; power-spectrum slope and navigability are more interpretable signals.

Why CPU mattered. The same architecture on a single mid-range GPU would have completed all seven unconditional runs in under an hour each (vs. 8–10 h CPU) and the cDCGAN in roughly 4–5 h. The 50-epoch ceiling was a direct consequence of CPU wall time; the lack of palette recovery and the mode collapse on three presets are both partly attributable to undertraining.

5 Conclusion

We trained seven WGAN-GP terrain generators on a synthetic Perlin-noise dataset and analyzed their behavior across a deliberately heterogeneous preset library. Training was stable but compute-limited; the resulting samples reproduce the broad spatial statistics of the data on roughly half the presets and exhibit single-feature mode collapse on the rest. Quantitative evaluation (Section **??**) confirms these qualitative findings: preset_07 achieves the best FID (340) and smallest power-spectrum deviation ($\Delta\beta = -0.29$), while the collapsed presets (03, 04, 05) show reachable-area fractions well below the real distribution (0.35–0.63 vs. 0.91–0.98). A planned conditional extension reached only epoch 14 of 50 in the available CPU time. The trained generators were successfully integrated into a real-time

A* navigation game (Section **??**) via a NumPy-only forward pass with snap-to-palette post-processing, demonstrating the end-to-end use case of the proposal.

Future work. The most productive next steps in roughly increasing scope are: (1) moving snap-to-palette (or a soft, differentiable approximation) into the training loss to address the smooth-interpolation artifact at the source; (2) replacing BatchNorm in G with LayerNorm or no-norm, which decouples samples within a minibatch and removes the documented WGAN-GP / BatchNorm-in-G instability; (3) longer training of the cDCGAN on GPU, which is the natural home for the "select terrain type at game-time" use case; (4) a categorical-output reformulation – predicting one of six palette tiers per pixel via a softmax head and a cross-entropy auxiliary loss – which would replace the regression-against-RGB problem with a much more natural classification problem for this data.

References

[1] I. Gulrajani, F. Ahmed, M. Arjovsky, V. Dumoulin, A. Courville. *Improved Training of Wasserstein GANs.* NeurIPS 2017.

[2] A. Radford, L. Metz, S. Chintala. *Unsupervised Representation Learning with Deep Convolutional Generative Adversarial Networks.* ICLR 2016.

Assistance from Claude AI.